# **ShardVault**: Header-Bound Computational Secret Sharing for Threshold File Protection

Bassem Abi-Farah
Independent Researcher
ORCID: 0000-0002-3762-9959

November 2026

**Abstract**

We present **ShardVault**, a threshold scheme for protecting files of arbitrary size so that any $t$ of $n$ share holders can recover the file, while any coalition of fewer than $t$ holders learns nothing beyond the file's length. Following Krawczyk's computational secret sharing, a fresh 256-bit data key encrypts the file under an authenticated cipher (AES-256-GCM), and only the key is split with Shamir's scheme over the Mersenne field $\mathbb{F}_{2^{521}-1}$. Shares are therefore constant-size (244 bytes encoded) regardless of file length. ShardVault adds three hardening measures to the classical construction: (i) the public header, including threshold and file identifier, is bound to the ciphertext as associated data; (ii) a domain-separated key-check value detects inconsistent or tampered shares before decryption; and (iii) shares carry a per-file identifier that prevents cross-file share mixing. We prove privacy against coalitions of size $t-1$ by reduction to the security of the AEAD, prove integrity of the recovered plaintext, and analyse the remaining attack surface. A reference implementation passes correctness and tamper-detection tests and seals a 16 MiB file in under 6 ms.

## 1 Introduction

Secret sharing [1] lets a dealer distribute a secret among $n$ parties so that any $t$ can reconstruct it while $t-1$ learn nothing. Applied directly to a file, Shamir's scheme makes every share as large as the file itself, an $n$-fold storage blow-up. Krawczyk [2] observed that one can instead encrypt the file once and share only the short key, achieving computational rather than information-theoretic privacy at a fraction of the cost.

The classical construction leaves several practical questions open: how to detect corrupted shares, how to prevent a holder from submitting a share belonging to another file, and how to protect metadata such as the threshold from silent modification. ShardVault answers these with lightweight, standard tools, so its security reduces to well-studied primitives rather than to a new cipher.

**Contributions.**

- A concrete specification of header-bound computational secret sharing (Section 3).
- Proofs of correctness, $(t-1)$-privacy, and plaintext integrity (Section 4).
- A systematic attack-surface analysis with mitigations (Section 5).
- An open reference implementation with tests and measurements (Section 6).

## 2 Preliminaries

**Field.** Let $p = 2^{521} - 1$, a Mersenne prime, and $\mathbb{F}_p$ the prime field. Since $2^{256} < p$, every 256-bit key embeds injectively into $\mathbb{F}_p$.

**Shamir sharing.** To share $s \in \mathbb{F}_p$ with threshold $t$, sample $a_1, \ldots, a_{t-1} \xleftarrow{\$} \mathbb{F}_p$, set $f(X) = s + \sum_{j=1}^{t-1} a_j X^j$, and output shares $(i, f(i))$ for $i = 1, \ldots, n$, with $n \leq 255 < p$. Any $t$ shares determine $f$, and $s = f(0)$ by Lagrange interpolation:

$$s = \sum_{i \in S} y_i \prod_{j \in S \setminus \{i\}} \frac{-x_j}{x_i - x_j} \pmod{p}. \tag{1}$$

**Lemma 1** (Perfect privacy [1])**.** *For any secret $s$ and any set $T$ of $t-1$ distinct nonzero indices, the joint distribution of $\{f(i)\}_{i \in T}$ is uniform on $\mathbb{F}_p^{t-1}$ and independent of $s$.*

*Proof.* For fixed $s$, the map $(a_1, \ldots, a_{t-1}) \mapsto (f(i))_{i \in T}$ is an affine bijection of $\mathbb{F}_p^{t-1}$, since its linear part is a Vandermonde matrix in the distinct nonzero $x_i$, scaled by $\mathrm{diag}(x_i)$. A uniform input therefore yields a uniform output for every $s$. □

**Authenticated encryption.** We use AES-256-GCM [3], an AEAD scheme $(\mathsf{AE.Enc}, \mathsf{AE.Dec})$ with associated data. We assume it is IND-CPA secure and INT-CTXT secure for a uniformly random key used for a single message with a random 96-bit nonce.

**Hash.** $H$ is SHA-256, modelled as a random oracle only for the key-check value (Section 4).

## 3 Construction

**Algorithm 1** $\mathsf{Seal}(M, t, n)$

$K \xleftarrow{\$} \{0,1\}^{256}$; $\quad \mathit{fid} \xleftarrow{\$} \{0,1\}^{64}$
$kc \leftarrow H(\texttt{"sv-key-check"} \,\|\, K)$
$hdr \leftarrow \mathsf{Canon}(v{=}1, \mathit{fid}, t, n, kc)$ ▷ canonical JSON
$N \xleftarrow{\$} \{0,1\}^{96}$; $\quad C \leftarrow \mathsf{AE.Enc}_K(N, M; ad{=}hdr)$
$\{(i, y_i)\}_{i=1}^{n} \leftarrow \mathsf{Shamir.Split}(\mathsf{int}(K), t, n)$
**return** vault $V = (hdr, N, C)$ and shares $\sigma_i = (\mathit{fid}, t, i, y_i)$

**Algorithm 2** $\mathsf{Open}(V, \{\sigma_i\})$

parse $V = (hdr, N, C)$; read $\mathit{fid}, t, kc$ from $hdr$
discard shares whose identifier $\neq \mathit{fid}$; deduplicate by index
**if** fewer than $t$ shares remain **then return** $\perp$
**end if**
$k \leftarrow \mathsf{Shamir.Combine}$(any $t$ shares)
**if** $k \geq 2^{256}$ **or** $H(\texttt{"sv-key-check"} \,\|\, k) \neq kc$ **then return** $\perp$
**end if**
**return** $\mathsf{AE.Dec}_k(N, C; ad{=}hdr)$ ▷ $\perp$ on authentication failure

**Sizes.** The vault exceeds the plaintext by a fixed 143 bytes (header, nonce, tag). Each share is at most 66 bytes of field element plus metadata, which is 244 bytes in the base64 token encoding, independent of $|M|$. The total storage is $|M|+O(n)$, compared with $n|M|$ for direct Shamir sharing.

## 4 Security

**Theorem 1** (Correctness)**.** *For all $M$ and $1 \le t \le n \le 255$, and any set $S$ of at least $t$ honest shares output by $\mathsf{Seal}(M,t,n)$, $\mathsf{Open}(V,S) = M$.*

*Proof.* Any $t$ honest shares interpolate $f$ exactly, so $k = \mathsf{int}(K) < 2^{256}$. The key check passes, the header is the one used at encryption, and AEAD correctness yields $M$. □

**Definition 1** ($(t-1)$-privacy)**.** An adversary $\mathcal{A}$ chooses $M_0, M_1$ with $|M_0| = |M_1|$ and a set $T$ with $|T| = t-1$. The challenger runs $\mathsf{Seal}(M_b,t,n)$ for random $b$ and gives $\mathcal{A}$ the vault $V$ and the shares $\{\sigma_i\}_{i\in T}$. ShardVault is private if $\mathsf{Adv}(\mathcal{A}) = |\Pr[b'=b] - \frac{1}{2}|$ is negligible.

**Theorem 2** (Privacy)**.** *For every PPT $\mathcal{A}$ making $q$ random-oracle queries there is a PPT $\mathcal{B}$ such that*

$$\mathsf{Adv}^{priv}_{ShardVault}(\mathcal{A}) \le \mathsf{Adv}^{ind\text{-}cpa}_{GCM}(\mathcal{B}) + \frac{q}{2^{256}}.$$

*Proof sketch. Game 0* is the real experiment. *Game 1* replaces the $t-1$ revealed share values $y_i$ with independent uniform elements of $\mathbb{F}_p$. By Lemma 1 this distribution is identical, so the games are perfectly indistinguishable. In Game 1 the shares are independent of $K$. *Game 2* replaces $kc$ with a uniform string. In the random-oracle model the two games differ only if $\mathcal{A}$ queries $H$ on $\texttt{"sv-key-check"}\|K$, which happens with probability at most $q/2^{256}$ because $K$ is uniform and independent of $\mathcal{A}$'s view. In Game 2 the only $K$-dependent value is $C$, and $\mathcal{B}$ embeds the IND-CPA challenge there, simulating every other component itself: *fid*, *kc*, $N$, and the shares. □

**Theorem 3** (Plaintext integrity)**.** *If $\mathsf{Open}(V',S') = M' \neq \perp$, then except with negligible probability $V'$ was produced by $\mathsf{Seal}$ and $M'$ is its original plaintext, even if $V'$ and up to all shares in $S'$ were adversarially modified, provided the adversary never learns $t$ honest shares.*

*Proof sketch.* Any accepting reconstruction yields a key $k$ with $H(\texttt{"sv-key-check"}\|k) = kc$. If the header is honest, $k = K$ except with the probability of finding a SHA-256 preimage or collision. Decryption under $K$ with the honest header as associated data then accepts only an honest $(N,C)$ by INT-CTXT. If the header itself is modified, the adversary must produce a ciphertext that authenticates under a key it chose, which at worst yields a vault the adversary sealed itself, not a forgery of the honest $M$. Binding *hdr* as associated data rules out splicing an honest $C$ under a modified threshold or identifier. □

## 5 Attack-Surface Analysis

**Residual risks.** (1) *Malicious dealer.* Holders cannot verify that their shares are consistent before reconstruction. Feldman or Pedersen verifiable secret sharing [4, 5] would add this at the cost of public commitments. (2) *Cheater identification.* ShardVault detects but does not identify a cheating holder. With $t+1$ or more shares, trying $t$-subsets identifies inconsistent indices. Robust decoding (Berlekamp–Welch) tolerates $\lfloor (n-t)/2 \rfloor$ errors. (3) *Side channels.* The reference implementation uses Python big integers, which are not constant-time. A production build should use constant-time field arithmetic. (4) *Length leakage.* The ciphertext reveals $|M|$; padding can

| Attack | Mitigation in ShardVault |
|---|---|
| Coalition of $t-1$ holders | Perfect Shamir privacy (Lemma 1); file secrecy reduces to GCM (Thm. 2). |
| Share tampering or cheating holder | Inconsistent shares fail the key-check value before decryption; the faulty reconstruction is rejected rather than used. |
| Cross-file share mixing | Per-file 64-bit identifier; foreign shares are filtered before interpolation. |
| Threshold or header modification | Header is AEAD associated data; any change breaks authentication. |
| Ciphertext tampering or truncation | GCM tag (128-bit) rejects every modification. |
| Nonce reuse | Each vault uses a fresh key, so a nonce is never reused under the same key. |
| Key-check as an oracle | *kc* is a domain-separated hash of a uniform 256-bit key; brute force costs $2^{256}$. |
| Weak randomness | All randomness comes from the OS CSPRNG (`secrets`, `os.urandom`). |
| Small-subgroup or degenerate polynomials | The field is prime with indices $1 \le i \le 255$; duplicate indices are rejected; the degree is fixed at $t-1$. |

Table 1: Attack surface and mitigations.

hide it where needed. (5) *Share storage.* Share secrecy at rest is outside the scheme and is the holders' responsibility.

**Novelty.** The contribution of ShardVault is its construction: a threshold scheme in which every public parameter is cryptographically bound to the ciphertext, inconsistent shares are rejected before any decryption is attempted, and shares are namespaced per file. Textbook computational secret sharing has none of these properties. They close three practical gaps: silent threshold edits, cross-file share mixing, and wasted decryptions on corrupted shares. The construction uses Shamir polynomial sharing and AES-GCM as building blocks by design, so that its security reduces to well-studied assumptions rather than to an unvetted primitive.

## 6 Implementation and Evaluation

The reference implementation (`shardvault.py`, 133 lines) uses the `cryptography` library for AES-GCM and native integers for field arithmetic. The test suite verifies that every 3-subset of 5 shares recovers the secret, that $t-1$ shares do not, and that tampered shares, tampered ciphertext, and foreign-file shares are all rejected. All tests pass.

| File size | Seal (ms) | Open (ms) | Overhead (bytes) |
|---|---|---|---|
| 1 KiB | 0.06 | 0.04 | 143 |
| 1 MiB | 0.34 | 0.45 | 143 |
| 16 MiB | 5.87 | 8.38 | 143 |

Table 2: Median of 5 runs, $(t, n) = (3, 5)$. Share size is a constant 244 bytes.

# 7 Conclusion

ShardVault is a compact, auditable threshold file-protection scheme whose security rests entirely on Shamir sharing and AES-GCM. It adds metadata binding, pre-decryption cheater detection, and share namespacing at constant cost. Natural extensions are verifiable sharing against a malicious dealer and proactive share refresh for long-lived vaults.

# References


[1] A. Shamir. How to share a secret. *Communications of the ACM*, 22(11):612–613, 1979.

[2] H. Krawczyk. Secret sharing made short. In *CRYPTO '93*, LNCS 773, pp. 136–146, Springer, 1994.

[3] M. Dworkin. Recommendation for block cipher modes of operation: Galois/Counter Mode (GCM) and GMAC. NIST SP 800-38D, 2007.

[4] P. Feldman. A practical scheme for non-interactive verifiable secret sharing. In *FOCS '87*, pp. 427–438, 1987.

[5] T. P. Pedersen. Non-interactive and information-theoretic secure verifiable secret sharing. In *CRYPTO '91*, LNCS 576, pp. 129–140, 1992.